August 27, 2026

BDA Assignment 2 and 3 Solutions

Big Data Analytics (303105361) — Assignment 2 & 3 Solutions

Assignment 2: NoSQL Data Management

5 Marks Questions

1. Explain NoSQL databases and their advantages over traditional databases.
NoSQL (Not Only SQL) databases are non-relational systems designed to handle large volumes of unstructured, semi-structured, or rapidly changing data. Unlike RDBMS, they don't enforce a fixed schema, which makes them flexible for evolving data models where new fields can be added without altering an entire table structure. They're built from the ground up for distributed environments, so data can be spread across many servers rather than scaled up on a single powerful machine.

Advantages include horizontal scalability across commodity servers, high performance for large read/write workloads, schema flexibility, and better handling of distributed and replicated data. They're well suited for real-time web applications, IoT data streams, and Big Data workloads where relational joins and strict ACID compliance aren't the top priority, and where availability and speed matter more than perfect consistency at every instant.

2. Compare SQL, NoSQL, and NewSQL databases.

Feature	SQL (RDBMS)	NoSQL	NewSQL
Schema	Fixed, predefined	Dynamic/flexible	Fixed, predefined
Scalability	Vertical (bigger server)	Horizontal (more servers)	Horizontal
Consistency	Strong (ACID)	Eventual (BASE)	Strong (ACID)
Data model	Tables with rows/columns	Key-value, document, column, graph	Relational, but distributed

Feature	SQL (RDBMS)	NoSQL	NewSQL
Best for	Structured transactional data	Big Data, unstructured/semi-structured data	High-throughput OLTP at scale
Examples	MySQL, Oracle	MongoDB, Cassandra	Google Spanner, CockroachDB

SQL databases prioritize consistency and structure, NoSQL trades some consistency for scale and flexibility, and NewSQL tries to get the scalability of NoSQL while keeping the transactional guarantees traditional applications rely on.

3. Explain different types of NoSQL databases with examples.

- **Key-Value stores:** Data stored as key-value pairs where the value is opaque to the database. Extremely fast lookups by key, minimal query capability beyond that. Example: Redis, DynamoDB.
- **Document databases:** Store data as JSON/BSON-like documents where each document can have its own structure. Supports querying inside the document itself. Example: MongoDB, CouchDB.
- **Column-family stores:** Data organized in column families rather than rows, optimized for reading/writing large volumes of sparse data and analytical queries. Example: Cassandra, HBase.
- **Graph databases:** Store data as nodes and edges, purpose-built for traversing relationships efficiently. Example: Neo4j, ArangoDB.

Each type trades off flexibility, query power, and performance differently depending on the access pattern the application needs most.

4. Describe key-value and document data models in NoSQL.

The key-value model stores data as a simple dictionary of unique keys mapped to values, with no defined structure inside the value itself; the database treats the value as a blob. It's optimized for very fast reads/writes and simple lookups by key, but offers almost no querying capability beyond that key, which limits it to use cases like caching, session storage, and shopping carts.

The document model stores richer, semi-structured records (typically JSON/BSON) where each document can have its own fields and nested structures, unlike a rigid table row. Databases using this model can index and query inside the document itself, filter by nested fields, and update partial documents, offering far more flexibility than key-value stores while still avoiding the rigid schema of an RDBMS.

5. Explain Map-Reduce and partitioning in NoSQL databases.
Map-Reduce is a programming model for processing large datasets in parallel across a cluster of machines. The **Map** phase takes raw input data and transforms it into intermediate key-value pairs, running independently across many nodes at once. The **Reduce** phase then collects all values sharing the same key and aggregates them into a final result, such as a count, sum, or summary.

Partitioning (also called sharding) splits a large dataset across multiple nodes based on a partition key, so each node only stores and processes a subset of the data. This is what makes horizontal scalability possible in the first place, and it's also what allows Map-Reduce jobs to run in parallel across partitions, since each node can run its Map phase on the data it already holds without waiting on the others.

4 Marks Questions

1. Why is NoSQL required in Big Data applications?
Big Data applications deal with the "3 Vs": large Volume, high Velocity of incoming data, and wide Variety of formats, none of which traditional RDBMS handle efficiently at scale. NoSQL databases scale horizontally across cheap commodity hardware instead of requiring one increasingly expensive server, and they handle unstructured or semi-structured data without forcing it into a rigid schema first. This gives them the write and read throughput needed for real-time analytics, social media feeds, sensor data, and other workloads where traditional joins and transactions would become a bottleneck.

2. Explain aggregate-oriented databases.
Aggregate-oriented databases group related data into a single self-contained unit called an aggregate, which is stored and retrieved as one piece rather than being normalized across multiple related tables the way an RDBMS would do it. For example, an order and its line items might be stored together as one document instead of being split into separate "orders" and "order_items" tables joined at query time.

This design makes it easy to distribute data across nodes since each aggregate is independent and doesn't need data from elsewhere to be understood, which also improves performance for read-heavy applications since a single read can fetch everything needed at once. Key-value, document, and column-family databases are all considered aggregate-oriented, while graph databases are the main exception.

3. Describe graph databases with suitable examples.
Graph databases store data as nodes (entities) and edges (relationships between entities) rather than tables, and both nodes and edges can carry their own properties. This makes

them ideal for data where the relationships matter as much as the data itself, such as social networks, recommendation engines, fraud detection networks, and knowledge graphs.

Example: in Neo4j, a "Person" node could connect to another "Person" node via a "FRIENDS_WITH" edge, and to a "Product" node via a "PURCHASED" edge. Traversing these relationships, like finding friends-of-friends or products bought by similar users, is fast in a graph database because it follows direct pointers rather than running expensive joins the way a relational database would need to.

4. Explain the advantages of NoSQL databases.
NoSQL databases offer horizontal scalability across commodity hardware, schema flexibility that lets data models evolve without downtime, and high performance for the specific access patterns they're designed around, whether that's key lookups, document queries, or graph traversal. They fit naturally into distributed, cloud-native architectures where data is replicated across regions for availability.

They also handle large-scale unstructured and semi-structured data well, and depending on the database and configuration, can be tuned to favor either high availability or strong consistency based on what the CAP theorem tradeoff demands for a given use case. This tunability is a major reason different NoSQL types dominate different industries.

5. Explain partitioning and combining in distributed databases.
Partitioning splits a dataset across multiple nodes, typically based on a hash or range of a partition key, so each node stores and processes only a portion of the total data. This improves scalability since adding more nodes increases both storage capacity and processing power, and it improves parallelism since queries and jobs can run across partitions simultaneously.

Combining refers to merging the results produced by these distributed partitions back into one unified output. This commonly happens as part of a Map-Reduce job, where each partition runs its own Map phase locally, and the intermediate results are then shuffled, sorted, and combined during the Reduce phase to produce a single coherent result across the entire dataset.

2 Marks Questions

1. **Define NoSQL.** A class of non-relational databases designed for flexible schemas and horizontal scalability, suited for large-scale unstructured or semi-structured data where rigid tables don't fit well.
2. **What is a key-value database?** A database that stores data as simple key-value pairs, offering very fast lookups by key with minimal query capability beyond that. Example: Redis.

3. **Define document database.** A NoSQL database that stores data as JSON/BSON-like documents with a flexible, self-describing structure, allowing fields to vary between documents. Example: MongoDB.
4. **What is a graph database?** A database that stores data as nodes and edges to represent entities and their relationships efficiently, optimized for traversal rather than tabular joins.
5. **Expand NewSQL.** NewSQL refers to a newer class of relational databases that combine the horizontal scalability of NoSQL systems with the strong ACID guarantees of traditional SQL databases.
6. **State any two advantages of NoSQL.** Horizontal scalability across commodity servers, and schema flexibility that allows the data model to evolve without downtime.
7. **What is Map-Reduce?** A programming model that processes large datasets in parallel using a Map phase that transforms data into key-value pairs, followed by a Reduce phase that aggregates them.
8. **Define partitioning.** The process of dividing a dataset across multiple nodes or servers based on a key, enabling distributed storage and parallel processing.
9. **What are aggregates in NoSQL?** A collection of related data treated as a single, self-contained unit for storage and retrieval, commonly used in document and key-value stores.
10. **Give two examples of NoSQL databases.** MongoDB (document store) and Cassandra (column-family store).

Assignment 3: Basics of Hadoop

5 Marks Questions

1. Explain Hadoop architecture with its components.
Hadoop follows a master-slave architecture built around two core layers: a storage layer (HDFS) and a processing layer (MapReduce/YARN). In the storage layer, the **NameNode** acts as the master, managing the file system namespace, directory structure, and metadata about which blocks live on which machines, while **DataNodes** act as slaves that store the actual data blocks and handle read/write requests from clients.

In the processing layer, **YARN** (Yet Another Resource Negotiator) manages cluster resources through a central **ResourceManager**, which allocates resources cluster-wide, and per-node **NodeManagers**, which manage resources on individual machines and run the actual tasks. This separation of storage and compute lets Hadoop scale each layer independently across large clusters of commodity hardware, and it's this design that gives Hadoop its fault tolerance and horizontal scalability.

2. Compare RDBMS and Hadoop.

Feature	RDBMS	Hadoop
Data type	Structured	Structured, semi-structured, unstructured
Scalability	Vertical (bigger server)	Horizontal (more nodes)
Schema	Schema-on-write	Schema-on-read
Cost	Expensive for large scale (licensing, hardware)	Cost-effective on commodity hardware
Processing	Centralized	Distributed and parallel
Fault tolerance	Depends on backups/replication setup	Built-in via block replication

RDBMS is a better fit when data is well-structured and transactions need strict consistency, while Hadoop is built for the scale and variety of Big Data where flexibility and cost matter more than instant consistency.

3. Explain the Hadoop Distributed File System (HDFS) architecture.
HDFS splits large files into fixed-size blocks (default 128 MB) and distributes them across a cluster of DataNodes rather than storing the whole file on one machine. Each block is replicated, by default 3 times, across different DataNodes so that the failure of any single node doesn't cause data loss, since the same block still exists elsewhere in the cluster.

The **NameNode** holds all the metadata, meaning file names, directory structure, and the mapping of which blocks belong to which file and where those blocks are located, but it never stores the actual file contents itself. When a client wants to read or write a file, it first contacts the NameNode to find out which DataNodes hold the relevant blocks, then transfers data directly to or from those DataNodes. This design lets HDFS handle very large files reliably and keeps the NameNode from becoming a bottleneck for actual data transfer.

4. Describe the workflow of Map Reduce in Hadoop.
A MapReduce job begins with the input data being split into fixed-size chunks, each of which is processed independently by a **Map** task running on the node where that chunk is stored, taking advantage of data locality to avoid unnecessary network transfer. Each Map task produces intermediate key-value pairs based on the logic defined by the programmer.

These intermediate pairs then go through a **shuffle and sort** phase, where data is transferred across the cluster so that all values for a given key end up grouped together and sorted on the same reducer node. The **Reduce** task then processes each group of values for a key and aggre-

gates them into a final output, which is written back to HDFS. The whole workflow runs in parallel across the cluster, coordinated by YARN, with failed tasks automatically retried on healthy nodes.

5. Explain shuffle and sort phases in Map Reduce.
After the Map phase produces intermediate key-value pairs on each node, the **shuffle** phase is responsible for transferring this data across the network so that all values associated with a given key, regardless of which Mapper produced them, end up on the same Reducer node. This step involves partitioning the Map output by key and moving data between machines, which makes it fairly network-intensive.

The **sort** phase then orders these keys, and the values within each key, before handing them off to the Reduce function, ensuring reducers always process data in a predictable, grouped order rather than in arbitrary arrival order. Because shuffle and sort involve significant disk I/O and network transfer across the cluster, this phase is often the most resource-intensive and time-consuming part of a MapReduce job, and it's a common target for performance tuning.

4 Marks Questions

1. What is Hadoop? Explain its key features.
Hadoop is an open-source framework for distributed storage and processing of large datasets across clusters of commodity hardware, originally developed as an Apache project. It's designed so that ordinary, relatively inexpensive machines can be combined into a cluster capable of storing and processing petabyte-scale data.

Key features include horizontal scalability, meaning capacity grows simply by adding more nodes; fault tolerance through automatic data replication across DataNodes; cost-effectiveness since it avoids expensive specialized hardware; and the ability to process structured, semi-structured, and unstructured data using a schema-on-read approach, where the structure is applied when the data is read rather than enforced when it's written.

2. Explain the history and need of Hadoop.
Hadoop originated from two influential Google papers describing the Google File System (GFS) and the MapReduce programming model, which inspired Doug Cutting and Mike Cafarella to build an open-source equivalent while working on the Nutch search engine project. It was later adopted and scaled at Yahoo before becoming a top-level Apache project used widely across the industry.

The need for Hadoop arose from the explosion of web-scale data that traditional RDBMS couldn't store or process cost-effectively, characterized by the "3 Vs" of Big Data: Volume (massive datasets), Velocity (data arriving continuously and fast), and Variety (structured, semi-structured, and unstructured formats mixed together). This demanded a distributed, fault-tolerant system that could run reliably on clusters of inexpensive hardware rather than requiring a single powerful and expensive server.

3. Describe the components of Hadoop ecosystem.
The core Hadoop ecosystem is built around **HDFS** for distributed, fault-tolerant storage, and **MapReduce/YARN** for distributed processing and resource management. Around this core sit several tools that make Hadoop more practical to use day to day: **Hive** provides a SQL-like query interface (HiveQL) over data stored in HDFS, **Pig** offers a scripting language for expressing data transformation pipelines, and **HBase** provides NoSQL columnar storage for random, real-time read/write access on top of HDFS.

For data movement, **Sqoop** transfers data between Hadoop and relational databases, while **Flume** ingests streaming data such as logs into HDFS. **Zookeeper** provides coordination services for distributed applications, handling things like configuration management and leader election across the cluster. Together these tools form the broader ecosystem that turns raw HDFS/MapReduce into a practical Big Data platform.

4. Explain input formats and output formats in Map Reduce.
Input formats define how raw input data is read and split into individual records for the Map phase. **TextInputFormat**, the default, reads data line by line and treats each line as a separate record, which works well for plain text logs. **SequenceFileInputFormat** instead reads binary, serialized key-value data, which is more efficient for intermediate data passed between MapReduce jobs. The input format also determines how a file gets split into chunks for parallel processing across Mappers.

Output formats define how the Reduce phase writes its final results back to storage. **TextOutputFormat**, the default, writes each key-value pair as a line of plain text, while other formats like **SequenceFileOutputFormat** write binary output suited for further processing by another MapReduce job. Choosing the right input/output format affects both performance and how easily downstream tools can consume the results.

5. Explain task execution in Map Reduce.
Task execution in MapReduce is coordinated by YARN rather than MapReduce itself. When a job is submitted, the **ResourceManager** allocates cluster-wide resources and launches an

ApplicationMaster specific to that job, which is responsible for negotiating containers, chunks of CPU and memory, from **NodeManagers** running on individual machines across the cluster.

Map and Reduce tasks then run inside these allocated containers, executing in parallel wherever the data and resources allow, with the ApplicationMaster continuously tracking progress and requesting more containers as needed. If a task fails or a node goes down mid-job, YARN automatically reschedules that task on a different, healthy node, which is a major part of what makes Hadoop resilient to hardware failures during long-running jobs.

2 Marks Questions

1. **Define Hadoop.** An open-source framework for distributed storage and parallel processing of large datasets across clusters of commodity hardware.
2. **Expand HDFS.** Hadoop Distributed File System, the primary storage layer of Hadoop.
3. **What is NameNode?** The master node in HDFS that manages file system metadata and block locations across the cluster, without storing actual file data itself.
4. **What is DataNode?** A slave node in HDFS that stores actual data blocks on disk and serves read/write requests from clients as directed by the NameNode.
5. **Define Map Reduce.** A programming model for processing large datasets in parallel using a Map phase that transforms data and a Reduce phase that aggregates it.
6. **What is shuffle in Map Reduce?** The process of transferring and grouping intermediate key-value pairs produced by Mappers so they reach the correct Reducer by key.
7. **Define input format.** A specification that determines how input data is read, split into records, and fed into Map tasks.
8. **Define output format.** A specification that determines how the final output produced by Reduce tasks is written to storage.
9. **State any two advantages of Hadoop.** Horizontal scalability by simply adding more nodes, and fault tolerance through automatic data block replication.
10. **What is task execution in Hadoop?** The process by which YARN's ResourceManager and NodeManagers allocate cluster resources and run Map/Reduce tasks inside containers across nodes, retrying failures automatically.

materio. originals

This document represents a printable version of the resource. Please note that the content may have been updated since this version was printed. For the most recent and accurate edition, we encourage you to scan the QR code provided on the front page to access the online version.

All notes and materials are curated by **InsightRoom**, a knowledge initiative under **materio.** Content is compiled from trusted and credible sources to ensure the highest standards of accuracy and quality. Where applicable, AI powered tools such as Perplexity may be used to assist in gathering verified insights from across the web.

To explore more curated insights and educational resources, please visit:
https://materioa.vercel.app/room

The InsightRoom